Tema 4: Components Lògics d'una Base de Dades

#bd #apunts #PARCIAL1

1. Esquemes (`CREATE SCHEMA`)

> **Definició d'Esquema**
>
> Un **esquema** és un component lògic que agrupa un conjunt definit i relacionat de **components de dades** (taules, vistes, dominis, índexs) i de **components de control** (procediments, disparadors, permisos).

Utilitat Administrativa

- Serveix com a **unitat administrativa centralitzada** per gestionar permisos, engegar, aturar o atorgar privilegis a un conjunt complet d'objectes sense haver de fer-ho individualment.
- Si una taula pertany a un esquema, les vistes, restriccions i disparadors associats normalment s'inclouen dins del mateix esquema.

Exemple SQL: Definició d'un Esquema

```sql
-- Creació d'un esquema lògic
CREATE SCHEMA organitzacio;

-- Creació d'una taula dins d'un esquema específic
CREATE TABLE organitzacio.departament (
    num_dept INT PRIMARY KEY,
    nom_dept VARCHAR(50) NOT NULL
);
```

---

2. Taules i Restriccions en DDL

Les taules es defineixen mitjançant el DDL (`CREATE TABLE`) incorporant restriccions de columna i de taula per garantir la integritat.

Tipus de Restriccions

- `NOT NULL`: Impedeix valors nuls a la columna.
- `UNIQUE`: Garanteix la unicitat dels valors (clau alternativa).
- `PRIMARY KEY`: Defineix la clau primària.
- `FOREIGN KEY ... REFERENCES`: Establix la integritat referencial amb accions compensatòries (`ON DELETE`, `ON UPDATE`) com `RESTRICT`, `CASCADE`, `SET NULL` o

`SET DEFAULT`.
- `CHECK (condicions)`: Garanteix que els valors compleixin una expressió o un rang del domini.

> ⚠ **Límits de les Restriccions `CHECK` en Taules**
>
> 1. Una restricció `CHECK` **no es viola mai si la taula està buida**.
> 2. Una restricció de taula/columna **només es comprova quan s'actualitza la taula on està definida**. Si una clàusula `CHECK` inclou una subconsulta a una altra taula externa, els canvis a la taula externa no reavaluaran la restricció.

Exemple SQL: Restriccions de Taula i Columna

```
CREATE TABLE empleat (
    nemp INT PRIMARY KEY,
    nom VARCHAR(50) NOT NULL,
    sou NUMERIC(8,2) CHECK (sou > 0),
    ndept INT,
    FOREIGN KEY (ndept) REFERENCES organitzacio.departament(num_dept)
    ON DELETE SET NULL
    ON UPDATE CASCADE
);
```

---

3. Assercions (`CREATE ASSERTION`)

> 🖊 **Concepte d'Asserció** Una **asserció** és una restricció d'integritat general que pot afectar **múltiples taules** o **múltiples files** d'una mateixa taula i que no es pot expressar de forma eficient com una restricció simple de columna o taula.

Característiques Principals

- A diferència de les restriccions `CHECK` de taula, les assercions **es comproven sempre** davant qualsevol modificació a la base de dades.
- En la majoria dels SGBD comercials actuals **no estan implementades directament** per motius de rendiment, de manera que s'acostumen a implementar mitjançant **disparadors (triggers)**.

Exemple SQL: Asserció entre Dues Taules

Requisit: Assegurar que tots els empleats treballin en un departament situat a la mateixa ciutat on resideixen.

```
-- Estructura de taules: empleat(nemp, ciutat_e, ndept) i dept(ndept,
ciutat_d)
```

```sql
CREATE ASSERTION ciutat_emp_dept CHECK (
    NOT EXISTS (
        SELECT *
        FROM empleat e, dept d
        WHERE e.ndept = d.ndept
          AND e.ciutat_e <> d.ciutat_d -- Not Equal
    )
);
```

---

4. Vistes (`CREATE VIEW`)

> ⓘ **Definició de Vista:** Una **vista** és una **relació derivada** l'esquema i contingut de la qual es calculen a partir d'altres relacions (bàsiques o derivades) mitjançant una consulta relacional (`SELECT`).

- La seva extensió **no existeix físicament** emmagatzemada a disc.
- És completament **consultable** i, en certs casos, **actualitzable**.

4.1. Definició i Consultes

Per definir una vista s'utilitza la sentència `CREATE VIEW`. De cara a l'usuari final, consultar una vista és transparent i equivalent a consultar una taula base.

Exemple SQL: Creació i Consulta de Vistes

```sql
-- Creació d'una vista filtrada
CREATE VIEW persones_manteniment AS
SELECT dni, nom, estudis
FROM persona
WHERE estudis >= 4;

-- OPTIONAL
CREATE VIEW nom_vista(option1, option2, ...) AS
...

-- Consulta transparent sobre la vista
SELECT *
FROM persones_manteniment
WHERE nom LIKE 'A%';
```

4.2. Actualitzabilitat de les Vistes i `WITH CHECK OPTION`

Les actualitzacions (`INSERT`, `UPDATE`, `DELETE`) realitzades sobre una vista es tradueixen automàticament en modificacions sobre la taula base.

Clàusula `WITH CHECK OPTION`

Garanteix que qualsevol inserció o modificació realitzada a través de la vista **compleixi la condició del** `WHERE` definit a la vista. Si l'operació faria que la tupla desaparegués de la vista, el SGBD **retorna un error i la rebutja**.

```
CREATE VIEW persones_qualificades AS
SELECT dni, nom, estudis
FROM persona
WHERE estudis >= 4
WITH CHECK OPTION;

-- Intent de modificació que incompleix el WHERE:
-- Intenta canviar 'estudis' a 2 (que és < 4)
UPDATE persones_qualificades SET estudis = 2 WHERE dni = 5;
-- ERROR: new row violates check option for view "persones_qualificades"
```

4.3. Quines Vistes són Actualitzables?

Perquè una vista sigui **actualitzable de forma transparent**, el SGBD ha de poder traduir l'operació a una **única acció no ambigua** sobre la taula de base.

Condicions de l'Estàndard per a Vistes Actualitzables

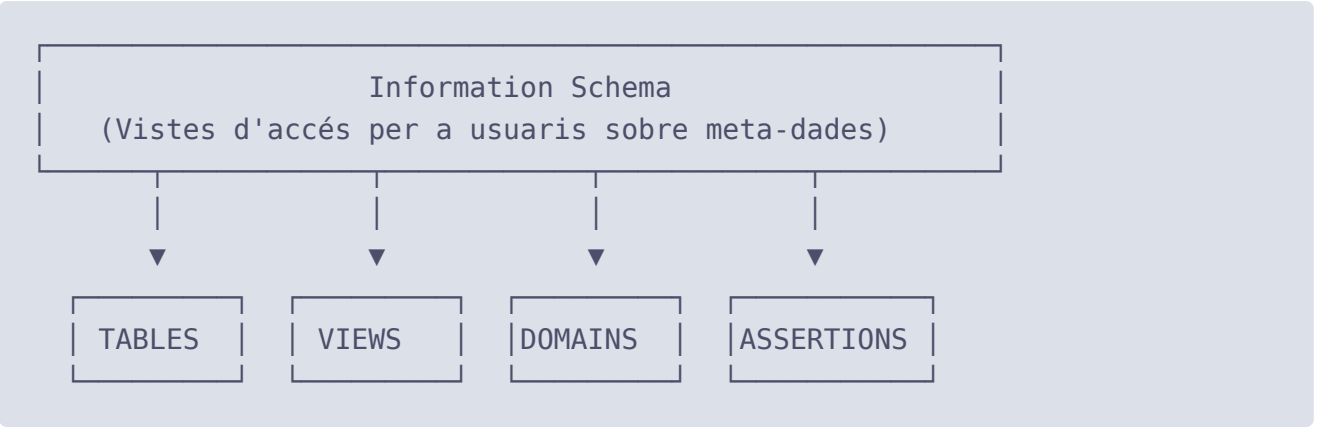
1. Definida sobre una **única relació base** (R).
2. Sense clausules d'agregació (`SUM`, `COUNT`, `AVG`) ni `DISTINCT`.
3. Sense clàusula `GROUP BY` ni `HAVING`.
4. La selecció de la vista ha d'incloure totes les columnes de la taula base que tinguin la restricció `NOT NULL` sense valor per defecte.

Tipus de Vista	Operació sol·licitada	Traducció a Taula Base	Estat
Simple (1 taula, filtre `WHERE`)	`UPDATE` / `DELETE`	Modifica o elimina la tupla corresponent a la taula base.	**Admesa**
Amb agregats (`GROUP BY`)	`UPDATE` de la suma	Múltiples formes de distribuir l'increment entre les tuples originals.	**No Admesa**
Multi-taula (Join)	`DELETE` o `INSERT`	Múltiples opcions (eliminar d'una taula, de l'altra o de totes dues).	**Restringida / No Admesa**

---

5. Esquema d'Informació (`Information Schema`)

L'**Esquema d'Informació** és un conjunt de vistes de només lectura estàndard que emmagatzema les **meta-dades** de la base de dades (informació sobre els esquemes, taules, columnes, restriccions, etc.).



- **Vistes principals**:
  - `SCHEMATA`: Informació dels esquemes.
  - `TABLES` / `VIEWS`: Informació de les taules i vistes definides.
  - `DOMAINS`: Informació dels dominis personalitzats.
  - `ASSERTIONS` / `TRIGGERS`: Informació sobre les restriccions globals i disparadors.

---

6. Components Lògics de Control: Privilegis i Rols

6.1. Gestió de Privilegis (`GRANT` i `REVOKE`)

Permeten controlar l'accés i les operacions que poden executar els usuaris sobre els objectes.

- **Sentències principals**: `GRANT` (atorgar) i `REVOKE` (retirar).
- **Opció `WITH GRANT OPTION`**: Permet que l'usuari que rep el privilegi pugui al seu torn atorgar-lo a altres usuaris.
- **Cascada en Revocar (`CASCADE`)**: Quan es revoca un privilegi atorgat amb `WITH GRANT OPTION`, la revocació es propaga en cascada a tots els usuaris que l'havien rebut indirectament.

Seguretat Granular: Combinació de Privilegis i Vistes

Si es vol restringir l'accés d'un usuari a determinades columnes o files d'una taula, es crea una **vista específica** i se li atorga el privilegi només sobre la vista.

Exemple SQL: Privilegis i Vistes

```
-- Crear vista per permetre a l'empleat 'joan' veure només el seu sou
CREATE VIEW sou_joan AS
SELECT sou
```

```
FROM empleat
WHERE nemp = 'joan';

-- Atorgar accés exclusiu a la vista
GRANT SELECT ON sou_joan TO joan;
REVOKE SELECT ON empleat FROM joan;
```

6.2. Rols (`CREATE ROLE`)

Un **rol** és una agrupació de privilegis definida per gestionar de manera eficient els permisos de grups d'usuaris.

```
-- 1. Definició de rols per part de l'Administrador (DBA)
CREATE ROLE lector;
CREATE ROLE escriptor;

-- 2. Asignació de privilegis als rols
GRANT SELECT ON organitzacio.departament TO lector;
GRANT INSERT, UPDATE ON organitzacio.departament TO escriptor;

-- 3. Assignació del rol als usuaris
GRANT lector TO Joan, Anna;
GRANT escriptor TO Toni;

-- 4. Activació del rol per part de l'usuari
SET ROLE lector;
```

---

7. Disparadors (Triggers) NO ENTRA AL PARCIAL 1

ⓘ **Concepte de Disparador** Un **disparador** (trigger) és un mecanisme **actiu/reactiu** que s'executa automàticament al SGBD quan es produeix un esdeveniment concret (`INSERT`, `UPDATE` o `DELETE`) sobre una taula.

Usos Principals dels Disparadors

1. **Auditoria d'operacions**: Registrar modificacions sensibles (ex: canvis de sou) gravant l'usuari, l'hora i els valors antics/nous en una taula d'auditoria.
2. **Avisos i regles de negoci complexes**: Validar condicions que no es poden expressar amb un `CHECK` o restriccions dinàmiques.
3. **Manteniment automàtic de dades derivades**: Actualitzar atributs calculats o vistes materialitzades de forma automàtica.
4. **Implementació d'Assercions**: Reemplaçar les assercions no suportades pels SGBD.

Exemple SQL: Disparador d'Auditoria

```
CREATE OR REPLACE TRIGGER audit_sou_empleat
AFTER UPDATE OF sou ON empleat
FOR EACH ROW
BEGIN
    IF :NEW.sou <> :OLD.sou THEN
        INSERT INTO auditoria_sou (nemp, sou_antic, sou_nou,
data_modificacio, usuari)
        VALUES (:OLD.nemp, :OLD.sou, :NEW.sou, CURRENT_TIMESTAMP, USER);
    END IF;
END;
```